## BÖLÜM 1 : Analiz ve Tasarım Tercihleri

### Neden std::vector veya diziler kullanılmadı?

1.Normal C dizilerinin verimsizliği ve maliyeti

C dizilerinin boyutu derleme sırasında sabittir.Bir müzik çalarda yeni şarkı eklenip silinebilmesi gerekir.Sabit boyutlu diziler bu dinamik yapıya müsait değildir.Dizi kapasitesi dolduğunda daha büyük bir diziye geçilmesi ve elemanların tek tek kopyalanması gerekir.

2.std::vector Yapısının Verimsizliği ve O(n) Operasyonlar

std::vector dinamik bir dizidir.Bellek yapısı gereği elemanları bellekte ardışık bir şekilde tutar.Dinamik dizi olması şu operayonlarda verim kaybına yol açar:

a) Eleman silme:

Liste başından veya ortasından silinen bir şarkını,bellek sürekliliğini korumak için her indeksin bir sola kayması gerekmektedir.Eleman sayısı n olduğu için işlem O(n) karmaşıklığına sahip olur ve verimsizdir.

b) Araya eleman ekleme:

Listenin başına veya ortasına bir eleman eklendiğinde o noktadaki tüm elemanların sağa kaydırılması gerekir.(O(n)).

c) Bellek yeniden tahsisi:

Vector kapasitesi dolduğu takdirde arka planda 2 katı büyüklüğünde yeni bir bellek alanı ayırır ve mevcut tüm şarkıları oraya kopyalar.Bu durum da geecikmelere neden olur.

### Bu senaryo için en uygun liste birleşimi (ÇİFT YÖNLÜ DAİRESEL BAĞLI LİSTE)

Bu müzik çalar senaryosu için en ideal yapı, **Çift Yönlü Bağlı Liste** ile **Dairesel Bağlı Liste** yapılarının **birleşimi (Çift Yönlü Dairesel Bağlı Liste)** türüdür.

Neden bu yapı?

a) Çift yönlü yapının gerekliliği:

Önceki şarkıya geçiş: Tek yönlü bağlı listeede düğümler sadece next işaretçisine sahiptir ve geriye dönmek mümkün değildir.Önceki şarkıya geçmek için listenin başından itibaren tekrar arama yapmak yani O(n) kadar zaman alır.

Çift yönlü yapıda her düğümde previous işaretçisi olduğu için O(1) zamanında geriye dönülebilir.

b) Dairesel yapının gerekliliği:

Tekrar modu:

Normal bağlı listde son düğümün next işaretçisi nullptr gösterir.Tekrar modu açıkken son şarkıdan sonra ilk şarkıya dönmek için if-else kontrolleri gerekir ama dairesel yapıda son şarkının next'i direkt olarak ilk şarkının previne bağlı olduğu için döngü oluşturulur.

## Seçim Gerekçeleri ve Big-O Analizi

**Avantajlar:**

O(1) Zaman Karmaşıklığı ile İki Yönlü Gezinme:

Hem düğüm hem next hem de prev işaretçilerine sahip olması bir ssonraki veya bir önceki şarkıya geçmede kolaylık sağlar.Liste boyutu ne kadar büyürse büyüsün geçiş süresi O(1) zamanda gerçekleşir.

O(1)  Zaman Karmaşıklığı ile Kesintisiz Dairesel Geçiş:

Head'in bir previnin olması ve tail'in bir nextinin olması ve bunların birbiriyle bağlantılı olmaı O(1) zamanda baştan sona geçiş ya da sondan başa geçmeyi kolaylaştırır.

O(1)Zaman Karmaşıklığı ile Sona Ekleme:

Adres kancaları sayesinde shift yapmadan en sona O(1) sürede tamamlanır.

O(1) Zaman Karmaşıklığı ile Düğüm Silme:

Silinecek düğümün adresi bilindiğinde,shifting yapılmadı için aradan çıkarma işlemi O(1) zamanda gerçekleşir.

**Dezavantajlar:**

Arama İşlemi Maliyeti — O(n) Zaman Karmaşıklığı:

Direkt olarak indeks ile erişim olmadığı ve tek tek bakılacağı kontrol edileceği için O(n) zamanda silme işlemi gerçekleşir.

Bellek Maliyeti:

Her düğüm sadece şarkı verileri değil iki adet pointer tutmak zorundadır(next,prev).bu durum tek yönlü listelere veya dizilere göre bellek tüketimini arttırır.

Karmaşık İşaretçi Yönetimi:

Ekleme ve Silme işleri risklidir.Birbirinden farklı dört işareetçi bağlantısının doğru yönetilmesi gerekir.Hatalı bağlantı durumunda memory leak gerçekleşebilir veya sonsuz döngü riski olabilir.

## BÖLÜM 2 : Veri Yapıları ve Sınıf Mimarisi

Node sınıfı: Oynatma listesindeki her şarkıyı ve bu şarkıların bağlantıları için adresini saklayan temel yapıdır.Node* next bir sonraki şarkının bellek adresini tutarken Node*prev bir önceki adresi tutar.

İşaretçilerin (Pointer) Çalışma Mantığı Algoritma Blokları

A. İleri Yönlü Bağlantı (next):

[Düğüm 1: Dağlar dağladı beni] ---> (next adresi) ---> [Düğüm 2: Nothing else matters]

**Algoritma Karşılığı:** el1->next = el2;

B. Geri Yönlü Bağlantı (prev):

[Düğüm 1: Dağlar dağladı beni ] <--- (prev adresi) <--- [Düğüm 2: Nothing else matters]

**Algoritma Karşılığı:** el2->prev = el1;

C. Dairesel Bağlantı (Circular Linkage):

[Düğüm 3 (tail): Null] ---> (next adresi) ---> [Düğüm 1 (head):Dağlar dağladı beni ]

[Düğüm 1 (head): Dağlar dağladı beni ] <--- (prev adresi) <--- [Düğüm 3 (tail): Null]

Playlist sınıfı içindeki 4 temel değişken ise sınırları ve o anki çalışma modunu yönetir.

```
class Playlist {
    public:
        Node* head;
        Node* tail;
        Node* currentSong;
        bool isRepeat;
        //  contructor: liste ilk oluştuğunda her şey boş
        Playlist() {...}
        //destructor: liste yok edilirken düğümlerin tamamı siliniyor
        ~Playlist() {...}
```

Node* head:Oynatma listesinin ilk şarkısını adreste tutar.

Node* tail:Oynatma listesinin en son eklenen şarkısının adresini tutar.

Node* currentSong:O an çalmakta olan şarkı düğümünü gösterir.

bool isRepeat:Listenin dairesel döngü davranışını aktif veya pasif yapar.

### Bölüm 3: Metotlar ve Algoritma

1. Playlist() ve ~Playlist() (Constructor & Destructor)

```
Playlist() {
    head = nullptr;
    tail = nullptr;
    currentSong = nullptr;
    isRepeat = false;
}
//destructor: liste yok edilirken düğümlerin tamamı siliniyor
~Playlist() {
    if (head = nullptr) return;//liste kontrolü boş ise bişey yapma
    tail->next = nullptr;// dairesel yapıdan kurtulmak için son bağı koparma
    Node* temp = head;//kopya yollayarak tek tek bellekten silebilme
    //döngü geçici değişkenin kopyalarını tek tek siliyor
    while (temp!= nullptr) {
        Node* nextNode = temp->next;
        delete temp;
        temp=nextNode;
    }
    head = nullptr;
    tail = nullptr;
    currentSong = nullptr;
}
```

Yapıcı metot Playlist():Bellek hatalarını engellemek için head,tail,currentSong işaretçileri nullptr olarak değerlendirilir.Bool olan isRepeat ise false yapılır.

Yıkıcı metot ~Playlist:New ile ayrılan düğümlerin bellek sızıntısını engellemek için delete komutunun olduğu yerdir.tail->next=nullptr yapılarak zincir sonlandırılır.Ardından temp ile geçici kopya oluşturulur ve döngü içinde tüm düğümler silinir.

2. addSong() (Şarkı Ekleme Algoritması)

```
void addSong(string artist,string title) {

    Node* newNode=new Node( title: artist, artist: title);
    if (head == nullptr) {
        head = newNode;
        tail = newNode;
        currentSong = newNode;

        // dairsel bağ kurma
        head->next = newNode;
        head->prev = newNode;
    }
    //listede eleeman varsa sona ekleme
    else {

        tail->next = newNode; //eski sonuncunun nextine yeni düğümü bağla
        newNode->prev = tail; // yeni düğümün previne eski sonuncuyu bağla

        newNode->next = head; //yeni sonuncunun nexti head gösterir
        head->prev = newNode; // başın previ yeni yeeni taili gösteriyor
    }
    tail=newNode; // yeni tail newNode
}
```

Liste boşken(İlk düğüm):Liste boşsa eklenen düğüm hem head,hem tail,hem de currentSong olarak atanır.Dairesel bağı kurmak için Düğümün next ve prev işaretçileri doğrudan kendisini gösterecek şekilde hazırlanır.

Listede eleman varken(Sona ekleme):Yeni düğüm mevcut tail düğümünün arkasına eklenir (tail->next = newNode ve newNode->prev = tail).Ardından dairesel yapıyı korumak için yeni düğümün next'i head'i head'in prev'i ise yeni düğümü gösterecek şekilde güncellenir.Son adımda tail işaretçissi yeni düğümee kaydırılarak işlem O(1) karmaşıklıkla tamamlanır.

3. nextSong() ve prevSong() (Gezinme Mantığı)

```
void nextSong() {
    if (currentSong == nullptr) return;

    if (!isRepeat && currentSong == tail) {
        cout << "Listenin sonuna geldiniz. (Tekrar modu kapali)" << endl;
        return;
    }

    currentSong = currentSong->next;
    cout << "Sonraki sarki: " << currentSong->title << " - " << currentSong->arti
}

void prevSong() {
    if (currentSong == nullptr) return;

    if (!isRepeat && currentSong == head) {
        cout << "Listenin basindasiniz. (Tekrar modu kapali)" << endl;
        return;
    }

    currentSong = currentSong->prev;
    cout << "Onceki sarki: " << currentSong->title << " - " << currentSong->artis
}
```

Gezinme işlemleri currentSong işaretçisinin next veya prev yönünde bir adım kaydırılması esasına dayanır.

Eğer isRepeat == true ise sınır kontrolleri devre dışı kalır;dairesel bağ sayesinde tail üzerindeyken nextSong() çağırıldığında doğrudan head'e head üzerindeyken prevSong() çağırıldığında doğrudan tail e geçilir.

Eğer isRepeat == false ise listenin sınırları kontrol edilir. currentSong == tail durumunda ileri gidilmesi, currentSong == head durumunda ise geri gidilmesi engellenerek kullanıcıya uyarı mesajı verilir.

4. removeSong() (Şarkı Silme Algoritması)

```cpp
bool removeSong(string title) {
    if (head == nullptr) return false; // Liste boşsa direkt false dön

    Node* temp = head;
    bool found = false;
    bool firstStep = true; // Dairesel döngüyü başlatan bayrak

    // 1. ŞARKIYI ARAMA (Düz while ile)
    while (temp != head || firstStep) {
        firstStep = false; // İlk adımı geçtik

        if (temp->title == title) {
            found = true; // Şarkı bulundu!
            break;
        }
        temp = temp->next;
    }

    // Şarkı listede yoksa silmeden çık
    if (!found) return false;

    // 2. SİLME İŞLEMİ VE BAĞLANTILARI GÜNCELLEME

    // DURUM 1: Listede tek bir şarkı varsa ve o siliniyorsa
    if (head == tail && temp == head) {
        head = nullptr;
        tail = nullptr;
        currentSong = nullptr;
    }
    // DURUM 2: Listede birden fazla şarkı varsa
    else {
        // temp'in solundaki ve sağındaki elemanları birbirine bağlayıp temp'i a
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;

        // Özel işaretçileri (pointer) kaydırma kontrolleri:
        if (temp == head) head = temp->next;         // Silinen ilk şarkıysa hea
        if (temp == tail) tail = temp->prev;         // Silinen son şarkıysa tai
        if (temp == currentSong) currentSong = temp->next; // Silinen çalan şark
    }

    // 3. BELLEKTEN SİLME (Memory Leak Önleme)
    delete temp;
    cout << "'" << title << "' listeye veda etti ve silindi." << endl;
    return true;
}
```

Mantık ve Arama Aşaması : Dairesel bağlı listede sonsuz döngüye girmemek için firstStep bayrağı kullanılarak head'den başlanır. Düz while (temp != head || firstStep) döngüsüyle tüm şarkılar taranır. Şarkı adı eşleşirse arama durdurulur.
Bağların Aradan Çıkarılması : Silinecek düğüm (temp) bulunduğunda, solundaki düğümün next'i sağındaki düğüme, sağındaki düğümün prev'i ise solundaki düğüme kilitlenir (temp->prev->next = temp->next ve temp->next->prev = temp->prev). Böylece düğüm zincirden aradan çıkarılır.
Sınır ve İşaretçi Güncellemeleri: Silinen eleman head, tail veya o an çalmakta olan currentSong ise sistemin çökmemesi için bu işaretçiler komşu düğümlere güvenle kaydırılır. Listede tek eleman kalmışsa tüm işaretçiler nullptr yapılarak liste sıfırlanır.
Bellek Temizliği: Son aşamada delete temp komutu çalıştırılarak ilgili düğüm bellekten tamamen kaldırılır ve memory leak önlenir.

5. printPlaylist() (Ekrana Yazdırma Algoritması)

```
void printPlaylist() {
    if (head == nullptr) {
        cout << "Oynatma listesi bos" << endl;
        return;
    }

    cout << "\n--- OYNATMA LISTESI ---" << endl;

    Node* temp = head;
    int index = 1;

    cout << index << ". " << temp->title << " - " << temp->artist;
    if (temp == currentSong) cout << " <== calan sarki";
    cout << endl;

    temp = temp->next;
    index++;

    while (temp != head) {
        cout << index << ". " << temp->title << " - " << temp->artist;
        if (temp == currentSong) cout << " <== calan sarki";
        cout << endl;

        temp = temp->next;
        index++;
    }

    cout << "----------------------\n" << endl;
}
```

Dairesel bağlı listelerde son düğümün next işaretçisi nullptr yerine tekrar head'i gösterdiğinden, standart bir while (temp != nullptr) kullanımı sonsuz döngüye sebep olur.

Bu duruma çözüm olarak liste başı olan head düğümü döngü öncesinde manuel olarak ekrana basılır ve temp = temp->next yapılarak bir sonraki düğüme geçilir. Ardından while (temp != head) koşulu kullanılarak tarama başlatılır. Böylece liste elemanları sırayla yazdırılır ve gezgin işaretçi tekrar head adresine ulaştığında döngü güvenli bir şekilde sonlandırılır.

**Test kısmı ve çıktılar**

```
--- OYNATMA LISTESI ---
1. Neset Ertas - Daglar dagladi beni <== calan sarki
2. Metallica - Nothing else matters
3. Rammstein - Ich will
----------------------

Su an caliyor: Neset Ertas - Daglar dagladi beni
Sonraki sarki: Metallica - Nothing else matters
Sonraki sarki: Rammstein - Ich will
Listenin sonuna geldiniz. (Tekrar modu kapali)
tekrar modu: acik
Sonraki sarki: Neset Ertas - Daglar dagladi beni

--- OYNATMA LISTESI ---
1. Neset Ertas - Daglar dagladi beni <== calan sarki
2. Metallica - Nothing else matters
3. Rammstein - Ich will
----------------------


Process finished with exit code 0
```